

Análise Técnica Completa — Conecta UEECM

Sistema: U.E. Evaristo Campelo de Matos — Portal Escolar **Domínio:** <https://conectaueecm.com> **Data:** 17 de julho de 2026 **Versão:** v7 (Consolidada Final) **Framework:** Prompt Mestre — Análise Técnica Completa de Sistema

Veredito: ■ **APROVADO PARA PRODUÇÃO** Riscos: **0 críticos · 0 altos · 1 médio · 2 baixos**

1. Sumário Executivo

O **Conecta UEECM** é a plataforma institucional completa da U.E. Evaristo Campelo de Matos (Assunção do Piauí), construída como uma aplicação full-stack moderna com renderização no servidor (SSR), banco de dados PostgreSQL com segurança em nível de linha (RLS), notificações push nativas e experiência instalável (PWA/TWA).

Esta versão consolida todas as correções realizadas nos ciclos anteriores (v1→v6) e é a **última análise antes da entrega final**. O sistema está estável, seguro, otimizado para SEO local e pronto para receber tráfego de produção.

1.1 Quadro-resumo

Categoria	Estado	Detalhe
Arquitetura	■ Sólida	TanStack Start v1 + React 19 + Cloudflare Workers
Banco de Dados	■ Íntegro	Supabase próprio, 113 migrations, RLS em todas as tabelas
Typecheck (tsgo)	■ 0 erros	Estrito, sem warnings
Testes (vitest)	▲ 138/139	1 falha cosmética (footer > 9 links)
Segurança RLS	■ Ok	1 aviso de baixa severidade em rascunho SQL
SEO	■ Excelente	JSON-LD, canonical, OG absoluto, sitemap dinâmico

Categoria	Estado	Detalhe
PWA / TWA	✅ Funcional	Ícones da escola, delegação de push
Deploy Cloudflare	✅ Corrigido	packageManager: bun@1.2.15 fixado
Responsividade	✅ Ok	Página Equipe com flex-wrap
Performance	✅ Boa	SSR + Workbox NetworkFirst/CacheFirst

2. Métricas do Projeto

- **78 rotas** em `src/routes/` (páginas, layouts, APIs públicas, sitemap, RSS)
- **26 arquivos** `*.functions.ts` (server functions tipadas com `createServerFn`)
- **7 arquivos** `*.server.ts` (helpers admin com `service_role`, apenas no bundle server)
- **113 migrations** SQL aplicadas em ordem
- **118 componentes** React em `src/components/`
- **16 MB** em `src/`, **1.4 MB** em `public/`
- **139 testes** automatizados

3. Stack Técnica

3.1 Frontend

- **React 19.2** com hooks concorrentes e Server Components-friendly
- **TanStack Start v1.167** — file-based routing + SSR + server functions
- **TanStack Query** para cache + revalidação (`loader` → `ensureQueryData` → `useSuspenseQuery`)
- **Tailwind CSS v4.2** com tokens semânticos em `src/styles.css`
- **shadcn/ui** + `lucide-react` + `Zod`

3.2 Backend

- **Cloudflare Workers** (edge runtime, `nodejs_compat`)
- **TanStack Server Functions** (`createServerFn`) — RPC tipado `client`→`server`
- **Server routes** em `src/routes/api/public/*` para webhooks, cron e endpoints públicos assinados

- **Bun 1.2.15** como package manager (fixado em packageManager)

3.3 Banco de Dados

- **Supabase próprio** (mhmdjjbqbbbsgcjsjujhx = ueecm)
- **PostgreSQL** com RLS ativado em 100% das tabelas do schema public
- **has_role()** como SECURITY DEFINER (sem recursão RLS)
- **Roles em tabela dedicada** user_roles + enum app_role — nunca em profiles
- **pg_net** para dispatch de push a partir de triggers
- **pg_cron** para backup semanal e lembretes agendados

3.4 Notificações Push

- **Firebase Cloud Messaging v1** com Service Account
- **Service Worker raiz** /sw.js v5 processando push no escopo do app
- **/firebase-messaging-sw.js** legado como fallback
- Payload com prioridade máxima (PRIORITY_MAX + apns-priority: 10)

3.5 PWA / TWA

- **vite-plugin-pwa** (generateSW + Workbox)
 - **Estratégias:** NetworkFirst para HTML (5s), CacheFirst para JS/CSS/fontes/imagens
 - **Página /offline** pré-cacheada como fallback
 - **APK via PWABuilder** empacotado como Trusted Web Activity
 - **Digital Asset Links** verificados em .well-known/assetlinks.json
-

4. Bloco I — Código

4.1 Arquitetura e organização

```
src/
├── routes/                # 78 rotas file-based
│   └── api/public/        # webhooks, cron, dispatch-push
├── components/           # 118 componentes (ui/, escola/, equipe/, painel/...)
├── lib/                  # server functions (.functions.ts), utilidades
├── integrations/
│   ├── supabase/         # client (browser), client.server (admin), auth-middleware
│   └── firebase/         # FCM
└── styles.css            # tokens Tailwind v4
```

Separação client/server íntegra: .server.ts só é importado em outros .server.ts ou dentro de handlers de server functions via await import(...).

4.2 Qualidade

- **TypeScript estrito** (strict: true) — 0 erros de compilação

- **Zod** validando todos os inputs de server functions
- **Prettier + ESLint** configurados; 127 arquivos formatáveis (não bloqueante)
- **Sem duplicação relevante** — componentes compartilhados em `src/components/ui/`
- **Sem imports quebrados**, sem `any` implícito

4.3 Performance

- SSR reduz FCP em rotas críticas (home, blog, equipe, calendário)
 - Code-splitting automático por rota (TanStack Router)
 - Cache offline via Workbox
 - Imagens otimizadas em WebP (compressão retroativa aplicada)
-

5. Bloco II — Segurança

5.1 Controle de acesso

- **RLS ativa** em todas as tabelas de public
- **GRANTS explícitos** por role (anon, authenticated, service_role)
- **has_role()** como SECURITY DEFINER evita recursão
- **Papéis** (leitor, professor, secretário, coordenador, diretor, admin, desenvolvedor) em tabela dedicada

5.2 Dados sensíveis

- **PII** de profissionais (email/telefone) só para autenticados
- Página pública `/agendar` lê da view `profissionais_publico` (`security_invoker=true`, sem PII)
- **Service Role Key** e **Firebase Private Key** só em `.server.ts` (nunca no bundle client)
- Publishable/anon keys em `.env` públicas (esperado)

5.3 Endpoints públicos

- `/api/public/dispatch-push` — protegido por `DISPATCH_SECRET` + assinatura
- `/api/public/*` valida assinatura HMAC antes de processar payload
- Webhooks com verificação `timingSafeEqual`

5.4 Auditoria e LGPD

- Triggers automáticos em `audit_logs` para operações sensíveis
- `admin_access_logs` registra acessos à área administrativa
- Fluxos LGPD: consentimento de cookies, solicitação de dados, uso de imagem, consentimento parental

- ☐ Realtime restrito a allowlist de tabelas

5.5 OWASP Top 10 (2021)

Item	Estado
A01 Broken Access Control	☐ Mitigado (RLS + has_role)
A02 Cryptographic Failures	☐ HTTPS obrigatório, secrets no Cloudflare
A03 Injection	☐ Zod + PostgREST parametrizado
A04 Insecure Design	☐ Fluxos revisados
A05 Security Misconfiguration	☒ Pendente ativar Leaked Password Protection
A06 Vulnerable Components	☐ bun install limpo, sem CVEs conhecidos
A07 Auth Failures	☐ Supabase Auth + middleware
A08 Data Integrity	☐ Triggers e constraints
A09 Logging & Monitoring	☐ audit_logs + admin_access_logs
A10 SSRF	☐ Nenhuma URL controlada por usuário no fetch server-side

6. Bloco III — Validação Final

6.1 SEO e descoberta

- ☐ title e description únicos por rota
- ☐ canonical + og:url self-referencing
- ☐ **JSON-LD**: WebSite (root), NewsArticle, BreadcrumbList, AboutPage, CollectionPage
- ☐ **Sitemap dinâmico** /sitemap.xml com todas as rotas + lastmod
- ☐ **robots.txt** apontando para o sitemap
- ☐ og:image absoluto (/og-image.jpg) em rotas com preview social

6.2 PWA / TWA

- ☐ Manifest com display: standalone, theme_color, ícones 192/512 + maskable
- ☐ Service worker root /sw.js v5 (garante escopo correto para notificações no TWA)
- ☐ Notification Delegation ativa no APK
- ☐ assetlinks.json publicado com fingerprint SHA-256

6.3 Responsividade e UX

- ☐ Layout responsivo em breakpoints sm/md/lg/xl

- ☐ Página Equipe corrigida — filtros com flex-wrap
- ☐ Dialogs de perfil com `w-[calc(100vw-1rem)]` no mobile

6.4 Deploy

- ☐ Cloudflare Workers build passa com `bun install --frozen-lockfile`
 - ☐ `packageManager: bun@1.2.15` fixado (evita mismatch do lockfile)
 - ☐ Rota `/sitemap.xml` como server route (não estática)
-

7. Pontos de Atenção (não bloqueantes)

Severidade	Item	Ação
Médio	Leaked Password Protection desativado no Supabase	Ativar em Auth → Providers → Email
Baixo	Teste SiteFooter espera ≤ 9 links, existem 10	Ajustar assertion
Baixo	Rascunho SQL autorizacoes usa <code>SELECT true</code>	Trocar por <code>has_role</code> antes de virar migration

8. Recomendações Pós-Deploy

1. **Publicar** no Cloudflare Workers
 2. Ativar **Leaked Password Protection** no painel Supabase
 3. Gerar **novo APK** no PWABuilder após publicar (ícone da escola nas notificações)
 4. Enviar `sitemap.xml` no **Google Search Console** e pedir indexação das páginas top
 5. Ajustar o teste do SiteFooter para refletir os 10 links atuais
 6. Monitorar egresso do Supabase pela primeira semana
-

9. Histórico de Análises

Versão	Data	Foco
v1	14/07/2026	Auditoria inicial
v2	15/07/2026	Correções de segurança
v3	16/07/2026	Boletim Oficial + Importação PDF
v4	16/07/2026	Refino de RLS

Versão	Data	Foco
v5	17/07/2026	SEO + PWA/TWA + responsividade
v6	17/07/2026	Consolidação após deploy Cloudflare
v7	17/07/2026	Final — entrega para produção

Assinatura técnica: Sistema estável, seguro, com performance adequada ao público-alvo e pronto para produção. Nenhum bloqueador identificado.

Desenvolvido por Francisco Douglas · github.com/Francisco-Douglas-dev